Lab Experiment 7: CI/CD Pipeline using Jenkins, GitHub and Docker Hub

Name: Mayank Thakur
Date: April 1, 2026
Subject: Containerization and DevOps

1. Aim

To design and implement a complete CI/CD pipeline using Jenkins, integrating source code from GitHub, and building & pushing Docker images to Docker Hub.

2. Objectives

- Understand CI/CD workflow using Jenkins (GUI-based tool)
- Create a structured GitHub repository with application + Jenkinsfile
- Build Docker images from source code
- Securely store Docker Hub credentials in Jenkins
- Automate build & push process using webhook triggers
- Use same host (Docker) as Jenkins agent

3. Theory

What is Jenkins?

Jenkins is a web-based GUI automation server used to:

- Build applications
- Test code

- Deploy software

It provides:

- Dashboard (browser-based UI)
- Plugin ecosystem (GitHub, Docker, etc.)
- Pipeline as Code using `Jenkinsfile`

What is CI/CD?

Continuous Integration (CI): Code is automatically built and tested after each commit.

Continuous Deployment (CD): Built artifacts (Docker images) are automatically delivered/deployed.

Workflow Overview

```
Developer → GitHub → Webhook → Jenkins → Build → Docker Hub
```

4. Prerequisites

- Docker & Docker Compose installed
- GitHub account
- Docker Hub account
- Basic Linux command knowledge

5. Part A: GitHub Repository Setup

5.1 Project Structure

```
my-app/
├── app.py
├── requirements.txt
├── Dockerfile
├── Jenkinsfile
```

5.2 Application Code — `app.py`

```python
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from CI/CD Pipeline!"

app.run(host="0.0.0.0", port=80)
```

5.3 `requirements.txt`

```
flask
```

5.4 Dockerfile

```dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 80
CMD ["python", "app.py"]
```

5.5 Jenkinsfile

```
pipeline {
    agent any
    environment {
        IMAGE_NAME = "your-dockerhub-username/myapp"
    }
    stages {
        stage('Clone Source') {
            steps {
                git 'https://github.com/your-username/my-app.git'
            }
        }
        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:latest .'
            }
        }
        stage('Login to Docker Hub') {
            steps {
                withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DO
                    sh 'echo $DOCKER_TOKEN | docker login -u your-dockerhub-username --
                }
            }
        }
        stage('Push to Docker Hub') {
            steps {
                sh 'docker push $IMAGE_NAME:latest'
            }
        }
    }
}
```

6. Part B: Jenkins Setup using Docker

6.1 `docker-compose.yml`

```yaml
version: '3.8'

services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
  jenkins_home:
```

6.2 Start Jenkins

```bash
docker-compose up -d
```

Access Jenkins at: `http://localhost:8080`

6.3 Unlock Jenkins

```bash
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

6.4 Initial Setup

- Install suggested plugins
- Create admin user (`MayankThakur`)

7. Part C: Jenkins Configuration

7.1 Add Docker Hub Credentials

Path: `Manage Jenkins → Credentials → Add Credentials`

- **Type:** Secret Text
- **ID:** `dockerhub-token`
- **Value:** Docker Hub Access Token

7.2 Create Pipeline Job

1. `New Item → Pipeline`
2. **Name:** `ci-cd-pipeline`
3. Configure:
    - Pipeline script from SCM
    - SCM: Git
    - Repo URL: your GitHub repo
    - Script Path: `Jenkinsfile`

8. Part D: GitHub Webhook Integration

8.1 Configure Webhook

In GitHub: `Settings → Webhooks → Add Webhook`

- **Payload URL:** `http://<your-server-ip>:8080/github-webhook/`
- **Events:** Push events

8.2 Exposing Jenkins via LocalTunnel

Since Jenkins runs locally, we used `localtunnel` to expose it publicly for GitHub webhook integration:

```
npm install -g localtunnel
npx localtunnel --port 8080
```

This generated a public URL (e.g., `https://green-donuts-nail.loca.lt`) which was used as the webhook payload URL.

9. Part E: Execution Flow

Stage	Action
Code Push	Developer pushes code to GitHub
Webhook Trigger	GitHub sends event to Jenkins
Clone	Jenkins pulls latest code from GitHub
Build	Docker builds image using Dockerfile
Auth	Jenkins logs into Docker Hub using stored token
Push	Image pushed to Docker Hub
Done	Docker image available globally

10. Role of Same Host Agent

Jenkins runs inside Docker with the Docker socket mounted:

```
/var/run/docker.sock
```

This allows Jenkins to directly control the host's Docker daemon — building and pushing images without needing a separate agent node.

A custom **Permanent Agent** node (`Mayank node macos`) was also configured in Jenkins under `Manage Jenkins → Nodes → New Node` to demonstrate multi-node agent setup.

11. Observations

- Jenkins GUI significantly simplifies CI/CD pipeline management
- GitHub acts as both source repository and pipeline definition store (via `Jenkinsfile`)
- Docker ensures consistent, reproducible builds across environments
- Webhook integration enables fully automated, event-driven pipelines
- `localtunnel` provides a quick way to expose local Jenkins to the internet for webhook testing
- The Docker socket mount allows Jenkins to control the host Docker engine directly

12. Result

Successfully implemented a complete CI/CD pipeline where:

- Source code and pipeline definition are maintained in GitHub
- Jenkins automatically detects code changes via GitHub webhook
- Docker image is built on the host agent
- Image is securely pushed to Docker Hub using stored credentials

13. Screenshots

Screenshot 1 — Jenkins Plugin Installation (Getting Started)

Jenkins Getting Started - Plugin Setup

> Jenkins loading suggested plugins during the initial setup wizard at `localhost:8080`.

Screenshot 2 — Create First Admin User

Create First Admin User

> Creating the admin user `MayankThakur` during Jenkins initial configuration.

Screenshot 3 — LocalTunnel Warning Page

LocalTunnel Tunnel Warning

> The localtunnel gateway page for `https://green-donuts-nail.loca.lt` — requiring IP confirmation before accessing Jenkins publicly.

Screenshot 4 — LocalTunnel Setup in Terminal

LocalTunnel Terminal Setup

> Terminal showing `npm install -g localtunnel`, `docker ps` confirming Jenkins container is running, and `npx localtunnel --port 8080` generating the public tunnel URL.

Screenshot 5 — LocalTunnel Connection Error (Firewall)

LocalTunnel Connection Error

> Localtunnel connection refused error due to firewall settings — resolved by switching to `npx localtunnel` instead of the global `lt` command.

Screenshot 6 — Jenkins Account Settings

Jenkins Account Settings

> Jenkins user account page for `mayankthakur` showing profile, credentials, and email settings via the localtunnel public URL.

Screenshot 7 — Adding New Node (Agent)

Jenkins New Node Creation

> Creating a new Permanent Agent node named `Mayank node` in Jenkins under `Manage Jenkins → Nodes → New Node`.

Screenshot 8 — Nodes Dashboard

Jenkins Nodes Dashboard

> Jenkins Nodes page showing the `Built-In Node` (Linux aarch64) and the newly added `Mayank node macos` agent.

Screenshot 9 — Agent Connection Command

Jenkins Agent Command

> Jenkins displaying the JNLP agent connection commands (Unix/Windows) with a secret token to connect `Mayank node macos` as a remote agent.

14. Viva Questions

Q1. What is the role of Jenkinsfile?
It defines the CI/CD pipeline as code, stored in the repository alongside the application source.

Q2. How does Jenkins integrate with GitHub?
Through GitHub Webhooks — GitHub sends a POST request to Jenkins on every push event, triggering the pipeline automatically.

Q3. Why is Docker used in CI/CD?
Docker ensures consistent, reproducible builds across different environments by packaging the application and its dependencies into an image.

Q4. What is a webhook?
A webhook is an HTTP callback that allows GitHub to notify Jenkins of events (like a push) in real time.

Q5. Why store Docker Hub token in Jenkins credentials?
To avoid hardcoding secrets in the Jenkinsfile, which is stored in a public/shared repository. Jenkins credentials store is encrypted and secure.

Q6. What is the benefit of using the same host as agent?

Jenkins can directly invoke Docker commands on the host, eliminating the need for a separate build agent and simplifying the setup.

15. Key Takeaways

- Jenkins is GUI-based but pipelines are fully code-driven via `Jenkinsfile`
- Always use the credentials store — **never hardcode secrets**
- Webhooks make CI/CD fully automatic and event-driven
- Docker socket mounting (`/var/run/docker.sock`) allows Jenkins to act as its own Docker agent
- `localtunnel` is a quick solution for exposing local services during development/testing

Experiment completed successfully on April 1, 2026.